Student Event Registration and Management System

Brief Documentation

1. Abstract

This project is a full-stack microservice-based platform for managing student event registrations and faculty-controlled event operations.
It implements secure authentication, role-based authorization, and responsive UI workflows suitable for academic mini-project evaluation.

2. Problem Statement

Manual event registration methods (forms/spreadsheets/messages) are error-prone and lack secure access control. This system provides a centralized and secure portal where students can view events while faculty admins can manage event records.

3. Objectives

1) Separate student and faculty workflows.
2) Implement JWT authentication with expiry.
3) Enforce role-based access control.
4) Secure passwords using BCrypt.
5) Provide clean and responsive frontend UI.

4. Technology Stack

Backend: Java 17, Spring Boot, Spring Security, Spring Data MongoDB, Spring Validation, Auth0 JWT, BCrypt.
Frontend: React, TypeScript, Vite, TailwindCSS, React Router DOM, Fetch API.
Database: MongoDB.

5. Microservice Architecture

Services:
1) student-service (Port 8082)
2) event-service (Port 8081)
3) faculty-service (Port 8083)

Layered structure in backend: controller -> service -> repository -> model/dto.
Inter-service communication: student-service fetches event data from event-service during login response composition.

6. Security and JWT Implementation

- Authentication: email + password.
- Passwords: BCrypt hash storage and BCrypt match verification.
- JWT contents: subject(email), role, issuedAt, expiresAt.
- Token is generated on login and sent to frontend.
- Frontend stores token and sends Authorization header: Bearer token.
- Backend JWT filter validates token and sets security context.
- Session policy: stateless.
- Protected endpoint access is controlled through role checks.

7. Role-Based Access Control

USER (Student): view own events only.
ADMIN (Faculty): view all events, add events, update events, delete events.

8. Features Implemented

- Student registration and login.
- Faculty registration and login.
- JWT-secured APIs.
- Role-aware dashboard behavior.
- Event CRUD controls for faculty admin.
- Student event viewing and profile block (Name, Roll No, Email).
- Click-to-edit event cards (admin).
- Toast messages, loading state, empty state, validation feedback.

9. Validation and Error Handling

Validation checks include required fields, email format, and password constraints.
DTO-based validation is used in backend request handling.
Global exception handlers return structured error payloads and proper HTTP statuses.

10. API Summary

Student Service

POST /student/register
POST /student/faculty/register
POST /student/login
GET /student/all

Event Service

GET /events/{rollNo}
GET /events (admin)
POST /events/add (admin)
PUT /events/{id} (admin)
DELETE /events/{id} (admin)

Faculty Service

POST /faculty/register
POST /faculty/login
GET /faculty/

11. Frontend Navigation Flow

Landing page -> Student/Faculty selection -> Register/Login -> JWT storage -> Protected events dashboard. (Dashboard rendering and permissions vary by role.)

12. Folder Structure (High Level)

backend/student-service:

config, controller, dto, exception, model, repository, security, service.

backend/event-service:

config, controller, dto, exception, model, repository, security, service.

backend/faculty-service:

controller, model, repository, security, service.

frontend/src:

assets, components, pages, routes, services, utils.

13. Configuration Summary

student-service: port 8082, studentdb URI, JWT settings, event-service URL.
event-service: port 8081, eventdb URI, JWT secret.
faculty-service: port 8083, facultydb URI.

14. Test Credentials

Student1 (Primary Student)

Name: Student
Email: student@ssn.edu.in
Password: stud@123
Roll No: 123
Role: USER

Student2
Name: Tushyent
Email: tushyent2410053@ssn.edu.in
Password: tush@123
Roll No: 189
Role: USER

Faculty1 (Primary Faculty Admin)
Name: Faculty1
Email: faculty@ssn.edu.in
Password: fac@123
Roll No: 9001
Role: ADMIN

Faculty2
Name: Kumar
Email: kumar@ssn.edu.in
Password: kumar@123
Roll No: 9002
Role: ADMIN

15. Seeded Event Data
Invente, Hackathon, AI Workshop mapped to Student (roll 123).
Tech Symposium, Startup Meetup mapped to Tushyent (roll 189).

16. How to Run
1) Start MongoDB.
2) Run event-service, student-service, faculty-service using Maven in separate terminals.
3) Run frontend (npm install, npm run dev).
4) Open http://localhost:5173 and login using Student1 or Faculty1 credentials.

17. Production Practices Included
Microservice separation, stateless JWT auth, BCrypt security, validation, role-based authorization, modular frontend, reusable components, and structured error handling.

18. Future Improvements

Add API gateway/service discovery, unified faculty-service security model, extended testing, and refresh token strategy.

19. Conclusion

The project demonstrates a complete secure full-stack implementation for campus event workflows with strong technical coverage across backend architecture, frontend UX, and access control.